



智能测试数据生成平台 —— 产品介绍文档

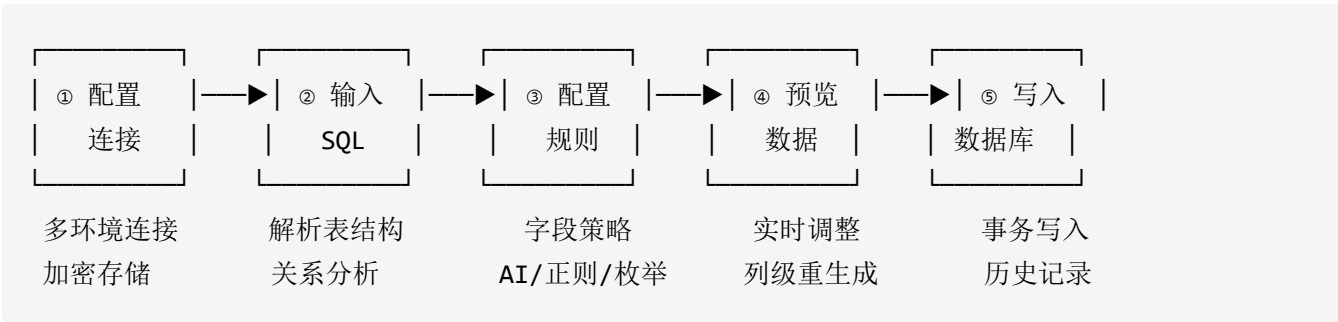
一、我们做了什么

基于 AI 大模型的智能测试数据生成平台：通过解析业务 SQL，自动理解数据库表结构与表间外键依赖关系，将 AI 大模型的语义生成能力与规则引擎的精确约束能力深度融合，一键生成既符合业务语义又满足所有数据库约束的真实感测试数据，并以事务方式直接写入目标数据库，同时完整记录任务执行快照供追溯复现。

二、功能方案设计

2.1 整体工作流程

平台采用「五步向导」设计，用户从 SQL 粘贴到数据落库全程零编码：



2.2 各步骤功能详解

Step 1 — 数据库连接管理

- 管理多套目标数据库连接（开发/测试/预发各套环境）
- 一键连通性测试，配置项 AES 加密存储
- 连接信息（host、port、username、database_name）统一管理，按需切换

Step 2 — SQL 输入与智能分析

输入支持：

- 手动粘贴业务 SQL (`INSERT INTO ... SELECT ...` 、多表 JOIN `SELECT`)
- 文件上传 (`.sql` / `.txt`) , 自动命名并保存到脚本库
- 脚本库：支持命名保存、重命名、删除，随时复用历史 SQL

自动分析内容 (SqlParserService)：

分析项	说明
涉及表清单	从 FROM、JOIN、子查询中提取所有表名
字段元信息	字段名、数据类型、最大长度、是否主键/自增/可空、注释
表间关系	JOIN 关联、数据库元数据中的 FK 约束，精确到字段级
拓扑排序	依 FK 依赖关系排出生成顺序，保证子表写入时父表已存在
WHERE 约束	自动提取 <code>WHERE / JOIN ON</code> 条件中的等值、IN、范围约束
循环依赖检测	出现循环 FK 时记录警告，原样追加以防死锁

WHERE 约束提取示例：

输入 SQL：

```
WHERE business_status IN ('ACTIVE', 'INACTIVE')
AND establish_date >= '2020-01-01'
AND establish_date <= '2024-12-31'
AND age > 18
```

自动推导出的规则建议：

- `business_status` → **枚举规则** `['ACTIVE', 'INACTIVE']`
- `establish_date` → **范围规则** `[2020-01-01, 2024-12-31]` (同列的 `>=` 与 `<=` 自动合并取交集)
- `age` → **范围规则** `min=18`

可视化展示：

- 关联关系图：以 `users.id → orders.user_id (FK)` 的连线方式展示
- 生成顺序：`users → orders → order_items`（拓扑箭头）
- 表结构详情：字段名、类型、PK/AI/NN 标记、注释、外键引用

Step 3 — 字段规则配置

四种生成策略：

策略	标识	配置项	典型用途
AI 语义生成	LLM_DESCRIPTION	语义描述文本	姓名、地址、公司名、订单备注等语义
正则表达式	REGEX	正则 pattern	身份证号、手机号、统一社会信用代码
范围值	RANGE	min、max、type	金额、年龄、日期区间、整数范围
枚举值	ENUM	候选值列表（可加权重）	状态码、性别、省份、业务类型

自动跳过字段： 自增列（`AUTO_INCREMENT`）和外键引用列（由引擎自动处理，不需用户配置）。

规则来源标记系统： 每条规则都带有来源标识，帮助用户理解建议来自哪里：

标记	颜色	来源
WHERE	黄色	SQL WHERE/ON 条件自动推导
HISTORY	蓝色	历史任务中相同字段的历史规则
KNOWLEDGE_BASE	紫色	知识库匹配建议
AI	紫色	大模型分析建议
COMMENT	灰色	字段注释自动识别
MANUAL	灰色	用户手动修改

规则配置辅助功能：

- **自动回填：** 页面加载时自动调用 API，将历史规则 + 知识库建议批量回填，大幅减少手工配置

- **一键 AI 建议：**针对整张表，调用大模型分析表结构（字段名、类型、注释、FK）后批量输出每列建议，直接回填到配置表单
- **字段历史规则：**点击任意字段可查看历史使用记录（规则类型、配置、使用次数、最近使用时间），一键应用

Step 4 — 数据预览与精细调整

预览数据表格：

行号	字段1	字段2(FK [∞])	字段3(AI [✦])	字段4(自增 [🔒])	...
1	张伟	10001	已发货	1	...
2	李娜	10003	待付款	2	...

表头功能：

- 复选框选中列 → 批量重新生成
- 字段属性图标：[🔒]（自增，不可编辑）、[∞]（外键，自动引用）
- 列菜单 (...)：重新生成此列 / 修改规则后重新生成 / 恢复原始数据

单元格编辑：双击进入内联编辑，Enter/失焦保存，Esc 取消，编辑后高亮标记"已修改"。

对比与恢复：开启 Diff 视图后，单元格下方显示"原: 旧值"；支持按列或整表一键恢复到初始预览状态。

列级重新生成（详见"功能亮点"章节）。

Step 5 — 写入数据库与任务记录

- 将预览数据（含所有手动编辑内容）批量写入目标数据库
- 全部表同一事务提交：任何一张表写入失败立即全部回滚，保证一致性
- 写入完成后自动创建任务历史记录，保存三类快照（规则 / 分析 / 数据）

任务详情页 Tab 结构：

Tab	显示条件	内容
库表结构	始终	交互式 ER 关系图（可拖拽）
SQL 语句	有输入 SQL	只读 SQL 查看

Tab	显示条件	内容
造数规则	始终	按表分组的字段规则快照
写入数据	成功任务	每表最多 200 行数据快照，横向滚动
错误日志	失败任务	保留格式的错误信息

2.3 功能亮点与创新点

亮点一：SQL 约束感知 — "SQL 即规则"

平台不只解析表结构，更深度分析 SQL 中的语义约束，将 `WHERE`、`JOIN ON` 条件反向转化为字段生成规则：

- **等值约束** `status = 'ACTIVE'` → 枚举规则 `['ACTIVE']`
- **IN 列表** `type IN ('A','B','C')` → 枚举规则 `['A','B','C']`
- **范围比较** `age >= 18 AND age <= 60` → 范围规则 `[18, 60]`
- **BETWEEN** `date BETWEEN '2024-01-01' AND '2024-12-31'` → 日期范围规则

同一字段的多个范围条件自动合并（取交集：最大下界 + 最小上界），避免生成不合规数据。

亮点二：大模型驱动的智能规则推荐

在规则配置页，用户可一键触发 AI 规则建议。平台构建包含表名、字段名、数据类型、字段注释、PK/FK/自增标记的结构化 Prompt，发送给大模型，由其分析每个字段最合适的生成策略，返回结构化 JSON 直接回填到配置表单。

大模型返回示例：

```
[
  {
    "columnName": "unified_code", "ruleType": "REGEX",
    "ruleConfig": {
      "pattern": "[0-9A-Z]{18}",
      "description": "18位统一社会信用代码"
    },
    "columnName": "company_name", "ruleType": "LLM_DESCRIPTION",
    "ruleConfig": {
      "description": "中国境内真实企业名称，含有限公司、股份公司等后缀"
    },
    "description": "企业名称"
  },
  {
    "columnName": "business_status", "ruleType": "ENUM",
    "ruleConfig": {
      "values": ["正常", "注销", "吊销", "迁出"]
    },
    "description": "企业经营状态"
  }
]
```

亮点三：列级无损重新生成

区别于"修改规则 → 全量重新生成所有列"的传统方案，平台支持**单列/多列精准刷新**：

操作流程：

用户在预览表头勾选要更换规则的列

↓

修改该列规则（或直接用菜单选"重新生成此列"）

↓

仅对目标列重新调用生成器（LLM / 正则 / 枚举 / 范围）

↓

其他列数据保持不变，FK 引用列从已有父表数据中匹配

↓

前端合并新列值到预览数据，触发 Vue 响应式更新

特殊处理：

- 自增列不在重新生成范围内（过滤跳过）
- 外键列从 `existingData` 中提取父表主键值注入，保证引用完整性
- 上下文感知： `context = {rowIndex, currentRow: existingRow}` 传递给生成器

亮点四：多表外键依赖全自动处理

当业务 SQL 涉及多张有外键关联的表时，平台端到端自动处理：

1. **关系发现**：从 SQL JOIN 关系 + 数据库元数据 FK 约束两路同时发现，精确到字段
2. **拓扑排序**（Kahn 算法）：计算出满足所有 FK 依赖的生成顺序
3. **同层并行**：同一依赖层级的表使用 `CompletableFuture` 并行生成，不同层级严格串行

- 4. **主键追踪**: 生成父表时追踪主键值到 `ConcurrentHashMap` , 子表 FK 列直接随机引用
- 5. **数据库兜底**: 若父表本轮未生成 (不在 SQL 范围内) , 自动从数据库查询已有值作为候选

亮点五：历史任务全量快照

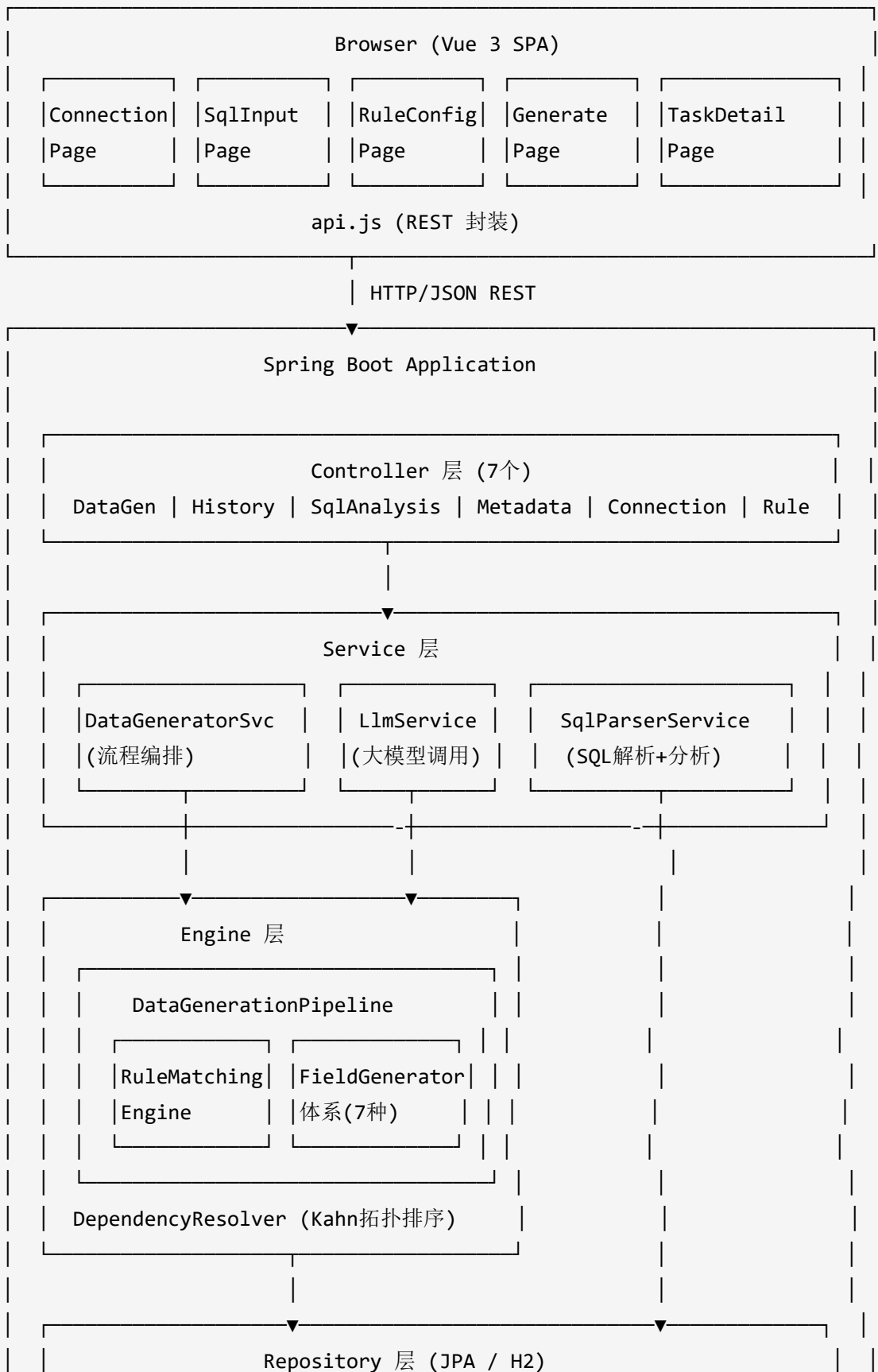
每次执行任务自动保存三类快照，任意时刻可完整回溯：

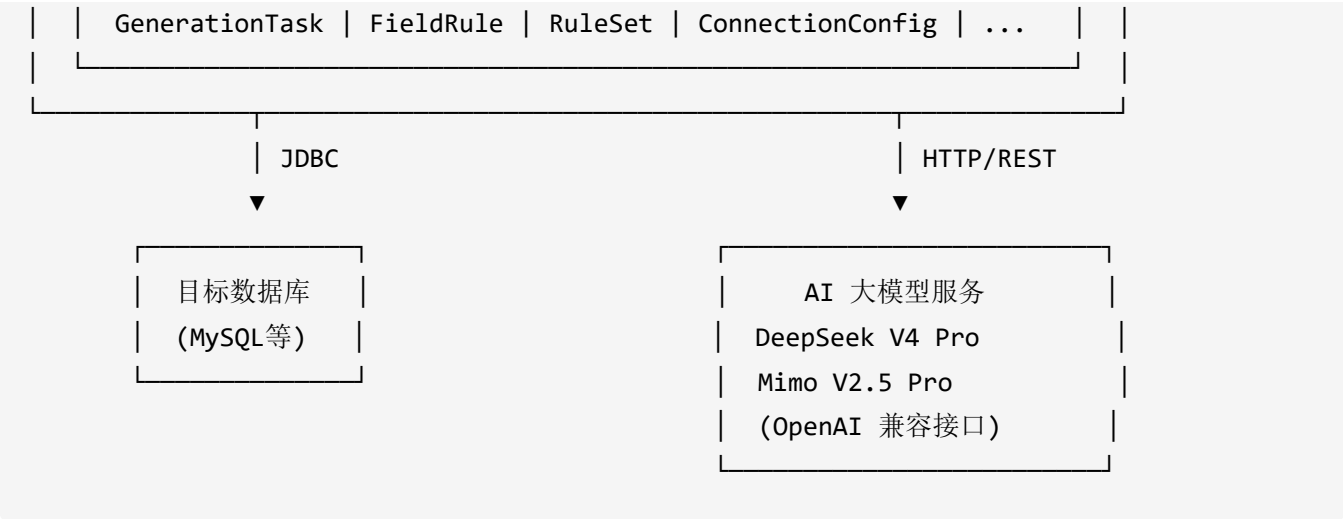
快照类型	存储字段	内容
规则快照	<code>rules_snapshot</code> (TEXT)	本次所有字段的生成规则配置 JSON
分析快照	<code>analysis_snapshot</code> (TEXT)	表结构、外键关系、生成顺序 JSON
数据快照	<code>generated_data_snapshot</code> (MEDIUMTEXT)	每张表写入的前 200 行数据 JSON



三、技术架构设计

3.1 系统整体架构





3.2 核心组件详解

3.2.1 数据生成引擎 (DataGenerationPipeline)

引擎是整个平台的核心枢纽，串联 SQL 分析结果、规则配置、LLM 服务和数据库写入。

三大核心方法：

① `preview()` — 纯预览（不写库）

`preview(DataGenRequest, SqlAnalysisResult) → DataPreviewResponse`

1. `buildDependencyLevels()`

└ 分析 FK 关系 → 返回分层列表 `[[table_A, table_B], [table_C], ...]`
同层表之间无依赖关系，可安全并行

2. 按层级处理（层间串行，层内并行）：

FOR each level in levels:

IF `level.size() == 1`: `generateTablePreviewData()` 串行

ELSE: `CompletableFuture.allOf(...runAsync())` 并行，等所有完成

3. `generateTablePreviewData(tableName, ...)`:

└ `ruleMatchingEngine.matchRules()` → generators (5级优先级)

└ `setupForeignKeyGenerators()` → 注入父表已生成的 PK 值

└ `preFillLlmGenerators()` → 并发预填充所有 LLM 字段

└ `generateRows(rowCount)` → 逐行调用 `generator.generate(context)`

4. 维护 `generatedPkMap` (`ConcurrentHashMap`)

→ 追踪生成的主键值，供后续层级子表 FK 引用

5. 按 `generationOrder` 排序结果

→ 返回 `DataPreviewResponse {tableData, generationOrder}`

② `execute()` — 生成 + 事务写入

```
execute(DataGenRequest, SqlAnalysisResult) → ExecutionResult
```

1. 同 preview 流程并行生成所有表数据

2. 按 generationOrder 排列写入顺序（严格顺序）

3. 事务写入：

```
conn.setAutoCommit(false)
```

TRY:

```
FOR each table in allTableData (按 generationOrder 顺序):
```

```
    dataWriterService.writeRowsOnConnection(conn, tableName, rows)
```

```
    trackGeneratedPks() → 追踪自增列生成的实际 key
```

```
    conn.commit()
```

CATCH Exception:

```
    conn.rollback() ← 任何表失败，全部回滚
```

4. 收集数据快照（每表最多 200 行）

→ 返回 ExecutionResult {rowCounts, snapshotData}

③ regenerateColumns() — 列级无损重生成

```
regenerateColumns(tableName, columns[], rowCount, fieldRules, models,  
                  analysisResult, existingData) → RegenerateColumnsResponse
```

1. 过滤自增列（加入 warnings，跳过）

2. ruleMatchingEngine.matchRules() → generators

3. setupForeignKeyGeneratorsFromExistingData()

└ 从 existingData 中提取父表 PK 值注入 ForeignKeyGenerator

4. preFillLlmGeneratorsForColumns() → 仅对目标列中的 LLM 列预填

5. FOR i = 0 to rowCount-1:

```
    context = {rowIndex: i, currentRow: existingData[tableName][i]}
```

```
    FOR each targetColumn:
```

```
        value = generators[col].generate(context)
```

```
        value = truncateIfNeeded(value, colMeta)
```

```
        columnData[col][i] = value
```

6. 返回 RegenerateColumnsResponse {columnData, warnings}

LLM 并发预填充机制：

```
preFillLlmGenerators(tableName, tableMeta, generators, totalRows, models)
```

收集所有 LlmBatchGenerator 类型的列

FOR each LLM列:

```
    CompletableFuture.runAsync(() -> {
        WHILE llmGen.needsMoreValues(needed):
            batchSize = min(llmBatchSize, needed)           // 默认批次 50 条
            description = 规则描述 || 字段注释 || 字段名
            values = llmService.generateBatchValues(...)
            IF values.isEmpty(): 用 DefaultGenerator 补充
            ELSE: llmGen.addValue(values)
    }, llmExecutor)    // 专用线程池: max(8, CPU×2)

CompletableFuture.allOf(...).join()    // 等待全部完成
CATCH: 回退到串行逐列预填
```

FieldGenerator 体系（7种实现）：

Generator	依赖	算法要点
LlmBatchGenerator	LlmService	预填值池（CopyOnWriteArrayList）+ 循环取用（AtomicInteger），线程安全
RegexGenerator	RgxGen 2.0	将正则表达式反向生成匹配字符串
RangeGenerator	—	按 type（int/long/double/date/datetime）在 [min, max] 内均匀随机
EnumGenerator	—	按权重数组进行加权随机抽样
ForeignKeyGenerator	—	从父表 PK 值池中 ThreadLocalRandom 随机引用
SequenceGenerator	—	从起始值按步长递增
DefaultGenerator	DataFaker 1.9.0	基于数据类型兜底生成（varchar/int/date/decimal/bool 等）

3.2.2 规则匹配引擎 (RuleMatchingEngine)

为每个字段绑定最合适的 `FieldGenerator`，采用 5 级优先级策略：

```
matchRules(tableName, columns, userRules, ruleSetId)
```

优先级 1（最高）：外键列

```
IF col.referencedTable != null → ForeignKeyGenerator (FK 父键稍后由 Pipeline 注入)
```

优先级 2：用户规则（本次请求中传入的 `fieldRules`）

```
精确匹配: tableName.equalsIgnoreCase() && columnName.equalsIgnoreCase()
→ GeneratorFactory.create(ruleType, ruleConfig, dataType, maxLength, nullable)
```

优先级 3：存储规则（数据库中保存的规则，支持通配符模式）

```
glob 模式匹配: tablePattern && columnPattern && dataTypePattern
* → .* ; ? → . （转换为正则，CASE_INSENSITIVE）
```

优先级 4：启发式规则（内置规则库，按列名/数据类型识别）

- └ 列名含 "email"/"mail" → REGEX: [a-z]{5,8}@(gmail|qq|163|outlook)\.com
- └ 列名含 "phone"/"mobile" → REGEX: 1[3-9][0-9]{9}
- └ 列名 == "status" → ENUM: ["0","1"] 权重 [0.3, 0.7]
- └ 列名 == "gender"/"sex" → ENUM: ["M","F"] 权重 [0.5, 0.5]
- └ 数据类型 starts_with enum(...) → 提取枚举值列表
- └ 列名含 "create_time"/"update_time" → RANGE: datetime [2024-01-01, 2025-12-31]

优先级 5（最低）：默认生成器

```
→ DefaultGenerator(dataType, maxLength, nullable)
```

3.2.3 SQL 解析服务 (SqlParserService)

基于 `JSQLParser 4.9` 对业务 SQL 进行深度语义解析：

解析层次：

输入 SQL

↓

CCJSqlParserUtil.parse(sql)

- ├ Insert 语句: 提取 target table + 内嵌 SELECT
- ├ Select 语句: PlainSelect / SetOperationList (UNION 等) / 子查询
- └ 解析失败: 正则回退提取表名

PlainSelect 解析流程:

- ├ FROM 子句: extractFromItem() → 表名 + 别名映射
- ├ JOIN 子句:
 - ├ getRightItem() → 右表 + 别名
 - ├ getOnExpressions() → 关联关系 + ON 条件提示
 - └ JOIN 类型: LEFT/RIGHT/INNER
- └ WHERE 子句: extractJoinRelation() + extractWhereHints()

WHERE 提示提取支持的表达式类型:

AndExpression / OrExpression → 递归处理左右子树

EqualsTo {col = val} → ENUM: [val] (过滤纯列间关联条件)

InExpression {col IN (...)} → ENUM: 提取 ExpressionList (过滤子查询 IN)

Between {col BETWEEN a AND b} → RANGE: {min: a, max: b}

GreaterThan/GreaterThanEquals → RANGE: min

MinorThan/MinorThanEquals → RANGE: max

左右翻转处理: "5 > age" → effectiveOp 翻转为 "age < 5"

同列 RANGE 条件合并 (mergeRangeHints) :

- age >= 18 AND age >= 21 → 取最大下界 min = 21
- age <= 60 AND age <= 50 → 取最小上界 max = 50

拓扑排序 (DependencyResolver, Kahn 算法) :

构建有向图：

parent (被 FK 引用的表) → children (有 FK 的表)

inDegree[table] = 入度 (被依赖数量)

Kahn 排序：

queue = {inDegree == 0 的表}

WHILE queue not empty:

node = queue.pop() → result.add(node)

FOR each child of node:

inDegree[child]--

IF inDegree[child] == 0: queue.add(child)

循环依赖检测：

IF result.size() < tables.size():

剩余表追加到 result + warnings 中记录

3.2.4 可视化 ER 关系图 (SchemaGraph)

- 基于 HTML5 Canvas + 纯 JS 实现，无外部图形库依赖
- **贪心层次分配算法**：MAX_LAYERS=5, MAX_PER_LAYER=4，避免所有表横向铺满
 - 长 FK 链 (A→B→C→...→H) 自动合并相邻层，保证不超出最大层数
 - 单层超过 4 个表时水平拆分为子列 (MAX_SUB_COLS=3)
- **节点拖拽**：onCardHeaderMouseDown 鼠标事件驱动，鼠标悬停提示"拖拽可移动表位置"
- 外键连线动态渲染，颜色按层区分

3.3 AI 大模型交互设计

3.3.1 两种 LLM 调用场景

场景 A：批量字段值生成 (核心高频调用)

触发时机：用户在规则配置中为字段选择了 LLM_DESCRIPTION 策略，点击"预览"
调用时机：DataGenerationPipeline.preFillLlmGenerators()，预览开始前并发调用
每次调用：一次 API 请求返回 50 条值（llm-batch-size=50，可配置）

Prompt 工程细节：

System Prompt（每次调用都携带）：

"你是一个严谨的数据库测试数据生成专家，必须严格遵守字段定义的所有约束（类型、长度、可空性、边界值等），绝不生成任何违反约束的值。"

User Prompt（动态构造，示例）：

"请根据以下字段定义生成50条realistic且diverse的测试数据，仅输出JSON数组，不包含任何其他说明。"

Table: t_enterprise

Column: company_name

Data type: varchar(200)

Max length: 200（若varchar类型则每个值字符数≤该值；若数值类型则忽略）

Nullable: false

Description: 中国境内真实企业名称，含有限公司、股份公司等后缀

额外要求：

- 若varchar(n)且n<36，严禁生成带连字符的UUID（应改用无连字符的32位十六进制、字母数字组合、雪花ID或符合长度的业务编码）
- 若bigint/bigint unsigned，注意非负范围
- 数据可以不唯一，但尽量多样化，且贴近description的语义，不要刻意使用边界值，尽量不要构造接近边界值的数据
- 输出格式：["北京华夏科技有限公司", "上海腾讯信息技术股份有限公司", ...]"

场景 B：表级规则推荐（低频辅助调用）

触发时机：用户在规则配置页点击"AI建议规则"按钮
调用时机：RuleConfigPage → API.suggestRules()
每次调用：一次 API 请求分析整张表的所有字段，返回每字段建议

Prompt 工程细节：

User Prompt（动态构造）：

"Analyze this database table and suggest data generation rules for each column.

Table: t_enterprise

Comment: 企业信息表

Columns:

- id (bigint) [AUTO_INCREMENT] [PK]
- unified_code (varchar(18)) -- 统一社会信用代码
- company_name (varchar(200)) -- 企业名称
- business_status (varchar(20)) -- 经营状态
- establish_date (date) -- 成立日期
- user_id (bigint) [FK->t_user]

对于每个非自增字段，根据字段的数据类型和数据长度上限，给出最佳建议

Return JSON array: [{columnName, ruleType, ruleConfig, description}]

ruleType: REGEX (with pattern) | RANGE (with min/max/type) | ENUM (with values/weights)
| LLM_DESCRIPTION (with description)

请用中文回答，description字段请使用中文描述。"

3.3.2 多模型支持与随机分配

模型配置（application.yml）：

```

app:
  openai:
    # 全局默认配置
    model: deepseek-v4-pro
    max-tokens: 10000
    temperature: 0.8
    timeout-seconds: 60
    enable-thinking: true      # 开启 Extended Thinking

    # 各模型独立配置（可覆盖全局值）
    models:
      "[mimo-v2.5-pro]":
        base-url: https://token-plan-cn.xiaomimimo.com/
        api-key: ${MIMO_API_KEY}
        name: Mimo V2.5 Pro
      "[deepseek-v4-pro]":
        base-url: https://api.deepseek.com
        api-key: ${DEEPSEEK_API_KEY}
        name: DeepSeek V4 Pro

```

新增模型只需在 `models` 下添加条目，无需修改任何代码。

模型随机分配（`LlmService.pickModel`）：

```

// 用户可为同一任务选择多个模型
// 不同字段随机使用不同模型，提升生成结果多样性
private String pickModel(List<String> models) {
    if (models == null || models.isEmpty()) return openAiProperties.getModel();
    if (models.size() == 1) return models.get(0);
    return models.get(ThreadLocalRandom.current().nextInt(models.size()));
}

```

3.3.3 Extended Thinking（深度推理）支持

针对 DeepSeek 和 Mimo 两大模型系列的推理能力分别适配：

```
// DeepSeek 系列
if (actualModel.startsWith("deepseek") && enableThinking) {
    body.put("thinking", new JSONObject().put("type", "enabled"));
}

// Mimo 系列
if (actualModel.startsWith("mimo")) {
    body.put("thinking", enableThinking);
}
```

开启后模型先进行内部推理链再给出答案，对复杂约束场景（如字段间联动约束、多条件组合）生成质量更高。

3.3.4 容错与降级机制

异常场景	处理策略
API 调用失败	自动重试，最多 3 次，指数退避（1s → 2s → 4s）
全部重试失败	返回 <code>[]</code> ，LlmBatchGenerator 值池为空
值池为空时生成	回退到 <code>DefaultGenerator</code> 按数据类型兜底生成
并发预填异常	回退到串行逐列预填模式
响应含 Markdown	自动去除 <code>```json</code> 和 <code>```</code> 代码块标记

任意场景下都能输出完整行数据，不会因 LLM 异常中断整个生成流程。

3.3.5 LLM 调用完整链路图

前端 RuleConfigPage / GeneratePage

|
| ① 用户操作触发 (AI建议 / 预览按钮)

▼

DataGeneratorService.generatePreview()

|
| SqlParserService.analyze() → SqlAnalysisResult

|
| DataGenerationPipeline.preview()

|
| preFillLlmGenerators()

|
| CompletableFuture × N 个 LLM 字段 (并发)

▼

LlmService.generateBatchValues()

|
| buildGenerationPrompt() ← 构造含约束的结构化 Prompt
| pickModel(models) ← 随机选择模型

|

▼

POST /v1/chat/completions (OpenAI-Compatible API)

System: 严谨测试数据生成专家
User: 字段定义 + 约束 + 格式要求
model: deepseek-v4-pro / mimo-v2.5-pro
thinking: enabled (推理增强)

|

▼

parseValuesFromResponse()

去除 Markdown → JSON.parseArray()

|

▼

LlmBatchGenerator.addValue(values)

→ valuePool (CopyOnWriteArrayList)

|

▼

generateRows() 时循环取用

generator.generate(context)

3.4 技术选型

层次	技术 / 库	版本	选型理由
后端框架	Spring Boot	2.7.18	生产成熟，零配置快速启动
数据持久化	Spring Data JPA + H2	—	内嵌零配置数据库，DDL 自动维护 (<code>ddl-auto: update</code>)
SQL 解析	JSQLParser	4.9	工业级 Java SQL 语法解析器，支持复杂多层嵌套 SQL
AI 调用	RestTemplate (OkHttp)	—	兼容 OpenAI Chat Completions 接口，无厂商锁定，按模型独立配置
基础数据生成	DataFaker	1.9.0	多语言本地化假数据兜底 (姓名/地址/日期等)
正则反向生成	RgxGen	2.0	将正则表达式反向生成符合模式的字符串
JSON 处理	FastJSON	1.2.83	高性能 JSON 序列化，支持泛型 TypeReference
前端框架	Vue 3	—	Options API，无构建步骤，SPA 嵌入静态资源直接部署
并发	CompletableFuture + 自定义线程池	—	LLM 并发调用，专用线程池 <code>max(8, CPU×2)</code>

3.5 数据库设计要点

核心表： `generation_task` (任务记录 + 三类快照)

```
CREATE TABLE generation_task (  
  id BIGINT PRIMARY KEY AUTO_INCREMENT,  
  connection_id BIGINT,  
  input_sql TEXT, -- 原始业务 SQL  
  row_count INT, -- 目标行数  
  status VARCHAR(20), -- PENDING/RUNNING/SUCCESS/FAILED  
  error_message TEXT, -- 失败原因  
  rows_generated INT DEFAULT 0, -- 实际生成行数  
  started_at DATETIME,  
  completed_at DATETIME,  
  rules_snapshot TEXT, -- 规则配置 JSON 快照  
  analysis_snapshot TEXT, -- SQL 分析结果 JSON 快照  
  generated_data_snapshot MEDIUMTEXT -- 生成数据快照，每表 ≤200 行  
); -- MEDIUMTEXT 最大支持 16MB
```

`generated_data_snapshot` 格式 (示例) :

```
{  
  "t_user": [  
    {"id": 1, "name": "张伟", "email": "zhangwei@qq.com", "status": "1"},  
    {"id": 2, "name": "李娜", "email": "lina@163.com", "status": "0"}  
  ],  
  "t_order": [  
    {"id": 1, "user_id": 1, "amount": 299.00, "status": "已发货"},  
    {"id": 2, "user_id": 2, "amount": 1580.00, "status": "待付款"}  
  ]  
}
```

3.6 并发与性能设计

优化点	实现方式	效果
表间并行生成	同依赖层级表使用 CompletableFuture.allOf 并行	N 张无依赖表并行，时间从 O(N) 降至 O(1)
LLM 字段并发	每个 LLM	M 个 LLM

优化点	实现方式	效果
	字段独立异步任务，专用线程池	字段并发请求，不互相阻塞
批量 LLM 调用	每次请求 50 条值（可配置），循环取用	减少 API 调用次数，单次请求摊薄延迟
数据库批量写入	<code>insert-batch-size: 500</code> ，批量 INSERT	单次 500 行，减少数据库往返
线程安全数据结构	<code>ConcurrentHashMap</code> ， <code>CopyOnWriteArrayList</code> ， <code>AtomicInteger</code>	并发写入时无竞争冲突

四、总结

本平台将 **AI 大模型的语义理解能力** 与 **规则引擎的精确约束能力** 深度融合，解决了测试数据生成领域长期面临的核心矛盾：

- **真实感 vs. 约束合规**：大模型生成高质量语义数据 + 系统强制约束校验 + 长度截断兜底，两者互补而非对立
- **灵活性 vs. 一致性**：列级重新生成支持局部调整，外键依赖自动处理保证全局一致性
- **效率 vs. 易用性**：SQL 约束自动推导 + 历史规则回填 + AI 建议，将规则配置时间压缩至分钟级

适用场景：涉及多表联查的复杂业务 SQL、需要高真实感数据的接口测试、数据库外键约束密集的核心交易系统测试数据准备。